

HEART

, , , ,

Abstract

Abstract text.

Keywords:

1. Model

In this section, we present the proposed model, as illustrated in Fig. 1. This model integrates zero-shot extraction, Dynamic Context Truncation (DynamicCT), self-verified augmentation, and Adversarial Training (AT) modules. When processing a new task, we first preprocess the raw text and append hard prompts to construct standardized input sequences. Subsequently, an instruction-tuned Large Language Model (LLM) is utilized to extract candidate entities, which are then subjected to rigorous cross-verification combining high-dimensional feature vectors and an external knowledge graph to eliminate hallucinatory pseudo-entities. Next, we employ DynamicCT, which takes the target entity as an anchor and dynamically expands the truncation window driven by a neuro-symbolic dual engine to capture both global and local pathological logic information.

To address the extreme long-tail data distribution and feature noise in clinical records, we initially utilize the LLM to generate initial clinical assertion labels and monitor the extreme long-tail category distribution to identify minority classes with severely insufficient samples. Then, we leverage the LLM to perform targeted rewriting on these minority class samples and generate high-quality augmented data via a self-verification mechanism. Subsequently, the extracted dynamic contexts and the self-verified augmented features are jointly fed into the `chinese-roberta-wwm-ext` classifier. During the fine-tuning stage, specific perturbations are applied to the word embedding layer to generate adversarial examples. Combined with the Focal Loss, the adaptive adjustment of loss weights precisely shifts the optimization fo-

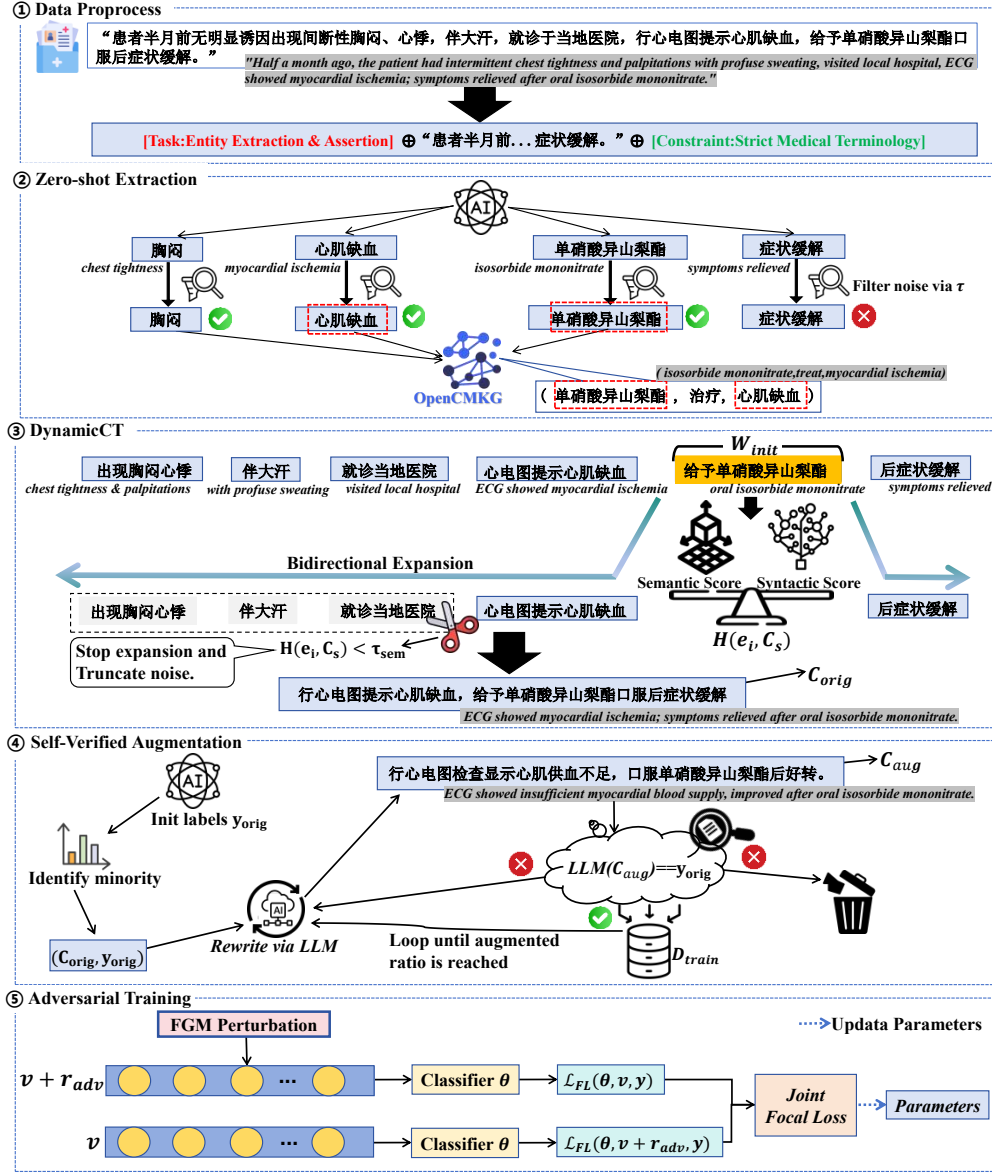


Figure 1: The overall model framework of DynaMed

cus toward hard samples, thereby achieving clinical assertion classification. The flowchart of DynaMed is presented in Fig. 2.

The fundamental steps of DynaMed are outlined in Algorithm 1. Lines

Table 1: Algorithm 1: Overall Training Process of DynaMed

Input: Raw text x , LLM M_{LLM} , Classifier θ
Parameter: Thresholds τ_{sem} & τ , Focal parameters α_t, γ_{focal} , FGM ϵ
Output: Optimized parameters θ^*

```

/* (1) Data Preprocessing */
1: for each sentence  $x$  do
2:    $x_p = T_{task} \oplus x \oplus T_{constraint}$ ;
3: end for

/* (2) Zero-shot Extraction */
4: for each preprocessed text  $x_p$  do
5:   Extract candidate entities  $E_{cand}$  via  $M_{LLM}$ ;
6:   for each  $e_i \in E_{cand}$  do
7:     if  $\max_{e_j \in E_{std}} \frac{h_{e_i} \cdot h_{e_j}}{\|h_{e_i}\| \|h_{e_j}\|} \geq \tau$  then
8:       Retain  $e_i$  and align with OpenCMKG for triples;
9:     else Filter out  $e_i$  as noise;
10:    end for
11: end for

/* (3) DynamicCT */
12: for each valid entity  $e_i$  do
13:   Set central clause as initial anchor  $W_{init}$ ;
14:   Expand bidirectionally based on hybrid score  $H(e_i, C_s)$ ;
15:   Terminate when  $\max H(e_i, C_s) < \tau_{sem}$ , output  $C_{orig}$ ;
16: end for

/* (4) Self-Verified Augmentation */
17: Identify minority samples  $(C_{orig}, y_{orig})$ ;
18: for each minority sample do
19:   Rewrite context:  $C_{aug} = M_{LLM}(C_{orig}, e_i, \text{prompt}_{rewrite})$ ;
20:   if blind-test prediction  $M_{LLM}(C_{aug}, e_i) == y_{orig}$  then
21:     Append  $(C_{aug}, y_{orig})$  to training set  $\mathcal{D}_{train}$ ;
22: end for
23: Down-sample majority classes in  $\mathcal{D}_{train}$  to balance data;

/* (5) Adversarial Training */
24: for each batch in  $\mathcal{D}_{train}$  do
25:   Extract continuous word embeddings  $v$ ;
26:   Compute FGM perturbation:  $r_{adv} = \epsilon \frac{\nabla_v \mathcal{L}_{FL}}{\|\nabla_v \mathcal{L}_{FL}\|_2}$ ;
27:   Compute joint Focal Loss:  $\mathcal{L}_{joint} = \mathcal{L}_{FL}(\theta, v, y) + \mathcal{L}_{FL}(\theta, v + r_{adv}, y)$ ;
28:   Update  $\theta$  via AdamW( $\nabla_{\theta} \mathcal{L}_{joint}$ );
29: end for
30: return  $\theta^*$ ;

```

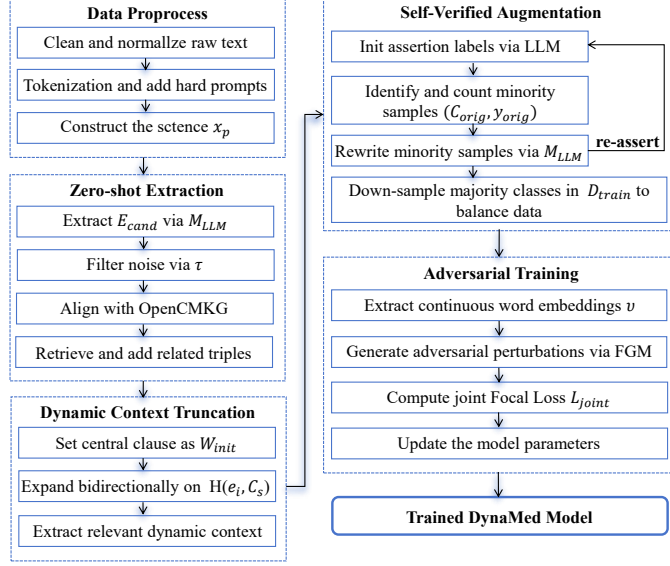


Figure 2: The overall training pipeline of DynaMed

1-3 cover data preprocessing, Lines 4-16 denote zero-shot extraction, Lines 17-20 describe the DynamicCT algorithm, Lines 21-29 delineate the self-verified data augmentation pipeline, and Lines 30-39 outline the adversarial training process. All mathematical symbols and formulas used in this section are summarized in Table 1.

1.1. Data Preprocessing

We first clean and standardize the raw data, which involves merging datasets, removing irrelevant symbols, and uniformly extracting plain text sequences from multi-source heterogeneous data. To assist the model in more accurately recognizing entities and relations, we adopt the hard prompt technique, injecting specific task instructions and constraint conditions into the input sentences to explicitly annotate entity types. We combine the input sentence x with the prompt template T to transform the sentence as follows:

$$x_p = T_{task} \oplus x \oplus T_{constraint} \quad (1)$$

where $\oplus$ denotes the sequence concatenation operation, T_{task} represents the task background description for entity extraction, and $T_{constraint}$ signifies the mandatory output format constraints. After completing these preprocessing

Table 2: Explanation of all the symbols in the formulas in this article.

Symbol	Explanation	Symbol	Explanation
x	Input sentence.	T	Prompt template.
E_{cand}	Candidate entity set extracted by LLM.	E_{std}	Standard entity set from OpenCMKG.
x_p	Preprocessed sequence.	h_{e_i}	Target entity vector.
$H(e_i, C_s)$	Hybrid scoring function for evaluation.	τ	Cosine similarity threshold for verification.
h_{e_j}	Standard entity vector.	W_{init}	Initial anchor window.
C_s	Candidate context fragment at step s .	D_{path}	Shortest path distance in dependency tree.
$\text{Sem}(\cdot)$	Semantic similarity.	$\text{Syn}(\cdot)$	Syntactic dependency.
$\mathbb{I}_{assert}$	Indicator function for assertion cues.	$\mathbb{I}_{ctx}$	Indicator function for context cues.
$\lambda_{critical}$	Critical cue reward.	δ	Cross-sentence penalty.
$\mathbb{I}_{cross}$	Firewall indicator for cross-clause boundaries.	α, β, γ	Weighting hyperparameters for mixed signals.
η	Distance decay factor.	C_{aug}	Augmented new context.
M_{LLM}	Parameterized LLM used for rewriting.	C_{orig}	Original dynamic context extracted.
v	Word embedding vector.	r_{adv}	Adversarial perturbation.
y_{orig}	Ground-truth assertion label of sample.	$\hat{y}$	Predicted verification label from LLM.
$\mathcal{L}_{FL}$	Focal Loss function.	p_t	Estimated probability.
ϵ	Scaling scalar controlling perturbation.	α_t	Weighting factor balancing class importance.

steps, the text data is utilized for subsequent model processing. This comprehensive preprocessing method not only improves and unifies the input format of multi-source data but also provides structural support for model training by introducing hard prompts, thereby significantly enhancing the model’s performance in clinical information extraction tasks.

1.2. Zero-shot Extraction

To alleviate the semantic hallucinations inherent in LLMs, we design a vector-calibrated zero-shot extraction method. Initially, we employ an

instruction-tuned LLM as a zero-shot extractor to derive a candidate medical entity set $E_{cand} = \{e_1, e_2, \dots, e_n\}$ from the preprocessed clinical text x_p .

However, general-purpose LLMs are prone to generating pseudo-entities that deviate from medical standards. To address this, we introduce a standard medical entity dictionary for factual verification. We construct a standard entity set E_{std} comprising all normative medical entities contained within the dictionary. Using the BGE high-dimensional vector model, we map each candidate entity e_i and the standard entity $e_j \in E_{std}$ from the dictionary into dense vector representations h_{e_i} and h_{e_j} . Subsequently, we construct a hard-threshold filter by calculating the maximum cosine similarity between them:

$$Sim(e_i, E_{std}) = \max_{e_j \in E_{std}} \frac{h_{e_i} \cdot h_{e_j}}{\|h_{e_i}\| \|h_{e_j}\|} \quad (2)$$

Only when $Sim(e_i, E_{std})$ is greater than or equal to a predefined threshold τ , is the candidate entity deemed factually compliant and retained. Finally, we align the filtered valid entities with an external clinical medical knowledge graph (OpenCMKG), retrieving their associated entity triples to introduce domain prior knowledge. This cascaded mechanism not only effectively filters out hallucinatory noise but also achieves deep enhancement of entity background information.

1.3. Dynamic Context Truncation

To preserve the long-range dependencies between entities and distant assertion cues in the Chinese context, we propose the DynamicCT method. By integrating syntactic dependency parsing and semantic similarity, this method dynamically expands outward using the target entity as an anchor, thereby accurately capturing the global and local semantic information of clinical logic.

First, we segment the raw text into several clauses (sense groups) based on punctuation marks and locate the central clause containing the target entity e_i to serve as the initial anchor window W_{init} . For contexts extending beyond the range of W_{init} , we employ a look ahead mechanism combined with a step-by-step expansion mechanism. Whether a candidate context fragment C_s is absorbed is governed by a hybrid scoring function $H(e_i, C_s)$, which deeply

integrates neural semantic relevance and symbolic syntactic dependency:

$$H(e_i, C_s) = \begin{cases} \lambda_{critical}, & \text{if } \mathbb{I}_{assert}(C_s) = 1 \\ \alpha \cdot \text{Sem}(e_i, C_s) + \beta \cdot \text{Syn}(e_i, C_s) + \gamma \cdot \mathbb{I}_{ctx}(C_s) - \delta \cdot \mathbb{I}_{cross}(C_s), & \text{otherwise} \end{cases} \quad (3)$$

where the indicator function $\mathbb{I}_{assert}(C_s)$ returns 1 when an explicit negative or positive assertion cue appears in the candidate fragment, thereby triggering the critical cue reward $\lambda_{critical}$ to forcefully absorb the fragment.

Under normal conditions, $\text{Sem}(e_i, C_s)$ represents the semantic cosine similarity between the entity e_i and the candidate fragment C_s ; $\text{Syn}(e_i, C_s)$ denotes the syntactic dependency score calculated via Spacy. To quantify the syntactic structure, this score is computed based on the shortest path distance D_{path} between the target entity e_i and the nearest assertion cue c_{cue} in the dependency tree:

$$\text{Syn}(e_i, C_s) = \max(0, 1 - \eta \cdot D_{path}(e_i, c_{cue})) \quad (4)$$

where η is the distance decay factor. $\mathbb{I}_{ctx}(C_s)$ is the context cue indicator function. Crucially, we introduce a cross-sentence penalty mechanism; when the expansion process crosses a main clause boundary (e.g., period, semicolon), the firewall function $\mathbb{I}_{cross}(C_s)$ returns 1 and applies a penalty coefficient δ to effectively prevent context contamination across different diseases. α, β, γ are weighting hyper parameters.

During the bidirectional left-and-right scouting at each step, the algorithm greedily selects the direction with the highest score to expand. The expansion process automatically terminates when the maximum score falls below the predefined semantic threshold τ_{sem} , or when the total length of the concatenated text reaches the maximum limit.

1.4. Self-Verified Augmentation

To tackle the severe class imbalance problem prevalent in clinical datasets, we initially utilize the LLM to generate initial clinical assertion labels and monitor the extreme long-tail category distribution to identify minority classes with severely insufficient samples. Subsequently, we employ the LLM to perform targeted rewriting on these minority class samples and generate high-quality augmented data via a self-verification (re-assertion) mechanism.

Specifically, given a minority class sample (C_{orig}, y_{orig}) containing the target entity e_i , where C_{orig} represents the original context refined by the

preceding module and y_{orig} denotes its initially generated ground-truth assertion label, we utilize a parameterized LLM (denoted as M_{LLM}) to generate a new augmented context C_{aug} :

$$C_{aug} = M_{LLM}(C_{orig}, e_i, \text{prompt}_{rewrite}) \quad (5)$$

where $\text{prompt}_{rewrite}$ is the guiding prompt designed to require the model to diversely rewrite the original syntactic structure under the absolute premise of preserving the core entity e_i and its pathological assertion logic.

However, when rewriting complex medical texts, large models are highly susceptible to “semantic drift” (e.g., inadvertently rewriting “negative” to “positive”). To ensure absolute label consistency of the augmented samples, we introduce a strict self-verification step. We feed the generated C_{aug} back into the model for a “blind test” (i.e., re-assertion) of its assertion status:

$$\hat{y} = M_{LLM}(C_{aug}, e_i, \text{prompt}_{verify}) \quad (6)$$

Only when the predicted label from the secondary verification perfectly matches the original label (i.e., $\hat{y} = y_{orig}$), is the augmented sample (C_{aug}, y_{orig}) considered high-quality data and appended to the final training set $\mathcal{D}_{train}$. Next, aggressively down-sample majority classes in $\mathcal{D}_{train}$ to balance data.

1.5. Adversarial Training

The final assertion classification is formulated as a sequence classification task. We input the extracted dynamic context $x_{dynamic}$ alongside the augmented data into the **chinese-roberta-wwm-ext** classifier. To enhance the model’s robustness against local redundant noise in clinical texts, we adopt an adversarial training method based on the Fast Gradient Method (FGM) during the fine-tuning stage.

The FGM algorithm applies a minor adversarial perturbation r_{adv} to the word embedding layer of the model (denoted as a continuous vector v) based on the gradient of the loss function:

$$r_{adv} = \epsilon \frac{g}{\|g\|_2}, \quad \text{where } g = \nabla_v \mathcal{L}_{FL}(\theta, v, y) \quad (7)$$

where ϵ is a scaling scalar controlling the perturbation magnitude, θ represents the model parameters, and y is the ground-truth label. By introducing perturbations within the continuous feature space, the model parameters θ

are optimized by minimizing the joint loss on both the original input and the perturbed input:

$$\min_{\theta} [\mathcal{L}_{FL}(\theta, v, y) + \mathcal{L}_{FL}(\theta, v + r_{adv}, y)] \quad (8)$$

To force the model to focus on hard-to-classify “hard samples” and further alleviate the extreme class imbalance, we replace the standard cross-entropy loss with the Focal Loss:

$$\mathcal{L}_{FL}(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (9)$$

where p_t denotes the model’s estimated probability for the ground-truth class. The focusing parameter $\gamma \geq 0$ is used to smoothly adjust the rate at which the weights of easy samples decay, thereby adaptively shifting the optimization focus toward minority hard samples; the weighting factor α_t is utilized to balance the inherent baseline importance of different classes.

References